Title of the Micro-Project

Predict stock prices using ordinary differential equations.

BY

S.NO	Students	Register Number
1.	Vidhyadhar	URK25CS7087
2.	Santhosa Priyan	URK25CS7100
3.	Malagari Sam	URK25CS7096
4.	Staines Balasingh	URK25CS7102

Introduction

To develop a stock price prediction model using **Ordinary Differential Equations (ODEs)** for continuous price evolution, enabling smoother predictions compared to traditional discrete models

Problem Statement

- **The Stock Market Prediction Challenge**

  Navigating the inherent volatility and randomness of financial markets.

- **Inherent Volatility & Randomness** :

  Stock prices are influenced by a myriad of unpredictable factors, leading to complex, non-linear movements.

- **Discontinuous Price Jumps** :

  Sudden, significant changes in price due to unexpected news or events.

- **Market Noise & External Factors** :

  Irrelevant data and macroeconomic events that obscure true price signals.

- **Limitations of Traditional Discrete Models** :

  Many models struggle to capture the continuous nature of price formation.

Objectives

- Build a user–item rating matrix for demonstration.
- Preprocess missing values using item-wise averages.

- Split observed ratings into training and testing sets.
- Apply SVD matrix factorization on training data.
- Reconstruct full predicted rating matrix.
- Evaluate model performance using RMSE.
- Display predicted rating matrix and error.

ODEs in Finance

- **Why Ordinary Differential Equations (ODEs)?**

  Leveraging the power of continuous mathematics for financial forecasting.

- **ODE**:

  Continuous math for financial forecasting.

- **Smooth Predictions**:

  Provides a continuous forecast curve.

- **Foundation for Advanced Models**:

  Basis for stochastic differential equations.

- **Models Continuous Change**:

  Captures instantaneous price movement

Mathematical Model

Theoretical Foundation: Ordinary Differential Equations

- **Theoretical Foundation: Ordinary Differential Equations**

Understanding the mathematical bedrock of continuous change.

An equation involving a function and its derivatives, describing how a quantity changes over time.

- **Classic Example: Population Growth**

$$\frac{dp}{dt} = kp$$

Here, dP/dt represents the rate of population change, directly proportional to the current population P, with S as the growth constant.

- **Financial Analogue: Stock Price Evolution**

$$\frac{dS}{st} = \mu S$$

- In this context, dS/dt signifies the instantaneous rate of stock price change, proportional to the current stock price S, with μ as the growth rate or drift

- **Our Stock Price ODE Model**

  A simplified model for continuous price evolution.

$$\frac{dS(t)}{dt} = \mu(t)S(t)$$

- **S(t)**: Stock price at time t.
- **dS(t)/dt**: Instantaneous rate of price change.
- $\mu(t)$:Time-dependent drift/growth rate.
- This model assumes price evolution follows continuous exponential growth based on historical drift, providing a foundational step towards more complex financial models.
- **Solving the ODE: Analytical Solution**

  Deriving the exact formula for stock price when drift is constant.

  1. Initial Equation: $\frac{dS}{dt} = \mu S$

  2. Separate Variables: $\frac{dS}{S} = \mu dt$

  3. Integrate Both Sides: $\frac{\int dS}{S} = \int \mu dt \Rightarrow \ln ln\,(S(t)) = \mu t + C$

  4. Solve for S(t): S(t) = $S_0 e^{ut}$

  Where S_0 is the initial stock price and μ^{ut} represents the continuous growth factor.

  This analytical solution provides a clear exponential growth trajectory under constant market conditions.

Methodology

- Our structured approach to building and validating the ODE-based prediction model.
- **Data Collection** :

  Fetch historical stock prices from reliable sources like Yahoo Finance.

- **Parameter Estimation** :

Calculate the time-dependent drift rate μ from historical returns.

- **Model Implementation** :

Develop the Euler method in Python to simulate price evolution.

- **Forecast Generation** :

Utilise the implemented model to predict future stock prices.

- **Validation** :

Compare predicted prices against actual market performance to assess accuracy.

.

Program (Code)

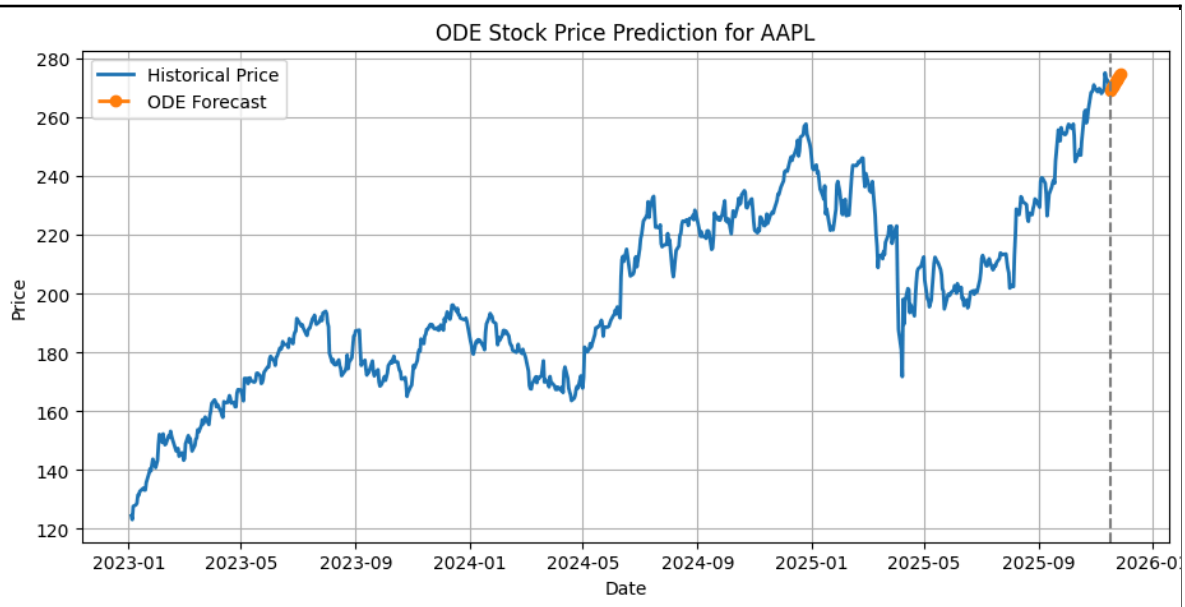
INPUT:

```
import yfinance as yf
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

# -------- SETTINGS --------
TICKER = "AAPL"        # Example: "RELIANCE.NS", "TCS.NS", "MSFT"
START = "2023-01-01"
FORECAST_DAYS = 10
# --------------------------

# 1. Download stock data safely
df = yf.download(TICKER, start=START, auto_adjust=True)

if df.empty:
    raise Exception("Error: No stock data found. Try a different ticker or date range.")

df = df[['Close']].dropna()

# Time index t = 0,1,2,...
df['t'] = np.arange(len(df))

# 2. Compute derivative dP/dt
df['dPdt'] = df['Close'].diff()
df = df.dropna()

P = df['Close'].values.flatten() # Ensure P is a 1D array
dPdt = df['dPdt'].values.flatten()
```

```
# 3. Fit ODE: dP/dt = aP + b using least squares
A = np.column_stack([P, np.ones(len(P))])
(a, b), *_ = np.linalg.lstsq(A, dPdt, rcond=None)

print("Estimated ODE Parameters:")
print(f"a = {a:.6f}")
print(f"b = {b:.6f}")

# 4. Numerical ODE model
def ode_model(t, P):
    return a * P + b

# Initial condition = last actual stock price
P0 = P[-1] # P0 is now a scalar

# Solve ODE over FORECAST_DAYS
t_span = (0, FORECAST_DAYS)
t_eval = np.arange(0, FORECAST_DAYS + 1)

sol = solve_ivp(ode_model, t_span, [P0], t_eval=t_eval)
forecast_values = sol.y[0]

# Create forecast dates
forecast_dates = pd.date_range(start=df.index[-1], periods=FORECAST_DAYS + 1,
freq='D')

# 5. Plot results
plt.figure(figsize=(11,5))
plt.plot(df.index, df['Close'], label="Historical Price", linewidth=2)
plt.plot(forecast_dates, forecast_values, '--o', label="ODE Forecast", linewidth=2)
plt.axvline(df.index[-1], color='gray', linestyle='--')
plt.title(f"ODE Stock Price Prediction for {TICKER}")
plt.xlabel("Date")
plt.ylabel("Price")
plt.grid(True)
plt.legend()
plt.show()

# 6. Show forecast table
forecast_df = pd.DataFrame({
    "Date": forecast_dates,
    "Predicted Price": forecast_values
})

print("\nForecasted Prices:")
print(forecast_df)
```

OUTPUT:

```
[**********************100%**********************]  1 of 1 completed
Estimated ODE Parameters:
a = 0.004656
b = -0.727493
```



Forecasted Prices:

	Date	Predicted Price
0	2025-11-17	269.480011
1	2025-11-18	270.008526
2	2025-11-19	270.539508
3	2025-11-20	271.072968
4	2025-11-21	271.608918
5	2025-11-22	272.147370
6	2025-11-23	272.688334
7	2025-11-24	273.231823
8	2025-11-25	273.777848
9	2025-11-26	274.326422
10	2025-11-27	274.877557

Applications

🎬 Our ODE model presents a compelling foundation for financial time series analysis, balancing elegant simplicity with practical utility.

🎬 It offers valuable insights into underlying trends, though its deterministic nature introduces certain limitations.

🎬 **Strengths**

- Captures overall trend well, providing a clear directional view.
- Smooth, continuous predictions, ideal for long-term outlooks.
- Computationally efficient, making it suitable for rapid analysis.

🎬 **Limitations**

- Ignores market volatility, understating risk in turbulent periods.
- Cannot predict sudden market crashes or abrupt shifts.
- Assumes a constant growth pattern (μ), which rarely holds true.

Insights

- Works best in stable, mature markets with predictable behaviours.
- Excellent for long-term trend analysis rather than short-term fluctuations.
- Serves as a strong foundation for building more complex, stochastic models.

Conclusion

ODE models provide a foundational approach to continuous-time financial modelling.

They offer smooth, mathematically elegant predictions for trend analysis, serving as an excellent starting point for understanding underlying market movements.

While simplified, their clarity makes them valuable for initial assessments and educational purposes.